

Dossier de conception technique

Écosystème K-Guard v1.4.0 : Laboratoire DevSecOps et Analyse SOC

Document de Référence pour le portfolio | Kamal Guidadou 2026

Sommaire

1. Introduction et Vision Stratégique

- 1.1 Contexte et Problématique
- 1.2 Objectifs du Projet
- 1.3 Positionnement : "Security by Design" et "SRE-First"

2. Infrastructure et Orchestration (Day 0 & Day 1)

- 2.1 Le Binaire Go : L'Ancre de Confiance
- 2.2 Processus de Provisioning : L'Idempotence et la Sécurité
- 2.3 Approche "Security by Design"

3. Architecture Backend et Sécurité Applicative

- 3.1 Un socle technique asynchrone (FastAPI & Pydantic)
- 3.2 Résilience SRE : Gestion de la Charge et Timeouts
- 3.3 Sécurité des Accès (Authentication & RBAC)
- 3.4 Sécurité de la "Supply Chain" logicielle

4. Module "K-Guard Network Sentinel"

- 4.1 Gouvernance et Micro-segmentation "Zero Trust"
- 4.2 Hardening Dynamique et Flux Chirurgicaux
- 4.3 Pilotage et Observabilité Réseau
- 4.4 Automatisation via Ansible

5. Observabilité et Maintenance (SRE)

- 5.1 Pilotage Opérationnel par le Dashboard "Settings"
- 5.2 Monitoring SRE : Prévention et Disponibilité
- 5.3 Maintenance Active : Gestion du Cycle de Vie des Données

- 5.4 Alerting SecOps : Intégration Cisco Webex

6. Conclusion et Posture Professionnelle

- 6.1 Synthèse de l'Architecture
- 6.2 Une réponse aux enjeux du marché
- 6.3 Posture d'Ingénieur et Perspectives

1. Introduction et Vision Stratégique

1.1 Contexte et Problématique

Dans l'écosystème actuel du Cloud-Native, la gestion d'un cluster Kubernetes (K3s) dépasse la simple mise en service de conteneurs. Elle impose des défis complexes en termes de maintien en condition opérationnelle (MCO), de sécurité de la *Supply Chain* logicielle et d'observabilité en temps réel. La multiplication des micro-services augmente mécaniquement la surface d'attaque, rendant les audits de sécurité manuels obsolètes et inefficaces face à la vélocité des menaces modernes.

K-Guard est né d'un constat opérationnel : le besoin d'une plateforme unifiée capable de transformer la sécurité d'un cluster d'un état passif et réactif à une posture d'**Active Defense** automatisée.

1.2 Objectifs du Projet

L'architecture de K-Guard a été conçue pour répondre à quatre impératifs critiques pour toute infrastructure de production :

- **Visibilité Totale (Observabilité SRE)** : Centraliser les *Golden Signals* pour garantir une disponibilité maximale et anticiper les défaillances (Capacity Planning, fuites de ressources, saturation de mémoire).
- **Gouvernance et Micro-segmentation (Zero Trust)** : Dépasser les limites des politiques réseau par défaut en instaurant un filtrage strict, autorisant uniquement les flux critiques et légitimes (filtrage par IP dynamique, isolation des namespaces).
- **Remédiation Automatisée (Self-Healing)** : Ne plus se limiter à l'alerte, mais agir. K-Guard intègre des mécanismes de *Smart Patching* et de rotation de pods pour corriger les vulnérabilités détectées (CVE HIGH/CRITICAL) sans interruption de service pour l'utilisateur final.
- **Intégration au Workflow SecOps** : Réduire le *Mean Time To Remediation* (MTTR) via une intégration native avec les outils de communication collaboratifs (Cisco Webex), permettant aux équipes SOC de recevoir des notifications contextuelles en cas d'intrusion ou de non-conformité.

1.3 Positionnement : "Security by Design" et "SRE-First"

K-Guard ne se présente pas comme un simple outil de monitoring, mais comme un laboratoire d'architecture Cloud-Native. Chaque composant, du binaire d'installation développé en Go à

l'orchestration des scans Trivy, a été pensé pour garantir :

- **L'idempotence** : Un déploiement fiable et reproductible.
- **La sécurité des droits** : Une gestion RBAC rigoureuse, respectant le principe du moindre privilège, avec des applications exécutées sans privilèges root.
- **L'évolutivité** : Une stack technique moderne (Vue.js, FastAPI, Go, SQLite) choisie pour sa performance et sa facilité de maintenance au sein d'un cluster léger.

2. Infrastructure et Orchestration (Day 0 & Day 1)

2.1 Le Binaire Go : L'Ancre de Confiance

Le déploiement de l'écosystème K-Guard n'est pas confié à de simples scripts Bash, mais à un binaire compilé en Go (v1.24.2). Ce choix technologique répond à deux objectifs majeurs : la portabilité et la sécurité dès le premier instant (*Security by Design*). L'interface terminale, propulsée par la librairie *Bubble Tea*, offre une expérience utilisateur intuitive tout en garantissant une exécution déterministe et sécurisée sur les environnements Linux cibles.

2.2 Processus de Provisioning : L'Idempotence et la Sécurité

L'installateur est conçu pour être idempotent, permettant des déploiements répétables sans dérive de configuration (*Configuration Drift*). Chaque étape du processus respecte les standards de durcissement (Hardening) attendus dans un environnement de production :

Étape	Action SRE	Détails Techniques et Justification
1. Diagnostic	Pré-flight Check	Validation de l'intégrité de l'OS et présence critique du fichier <code>/etc/rancher/k3s/k3s.yaml</code> pour l'accès API.

2. Nettoyage	Smart Cleanup	Exécution de kubectl apply --prune pour purger les ressources obsolètes et garantir un état propre (Clean State).
3. IAM	Sécurisation Admin	Hachage du mot de passe administrateur par l'algorithme fort Bcrypt (cost: 12) , garantissant une protection contre les attaques par force brute.
4. Environnement	Hardening Local	Génération du fichier .env avec des permissions strictes (chmod 600), incluant la gestion sécurisée des secrets (JWT, CORS, chemins d'accès).
5. Orchestration	Synchronisation K8s	Injection dynamique des secrets dans le namespace k-guard via un canal sécurisé, assurant que les credentials ne transitent jamais en clair.
6. Core Deployment	Application Manifeste	Déploiement automatisé des briques (Deployments, Services, RBAC, et Network Policies) pour assurer la cohérence de l'infrastructure.

2.3 Approche "Security by Design"

L'architecture de déploiement intègre des garde-fous pour prévenir les erreurs humaines et les mauvaises configurations :

- **Isolation des privilèges** : Dès la phase d'orchestration, K-Guard prépare le cluster pour que les applications tournent sous un utilisateur non-root (`runAsUser: 1000`).
- **Gestion des accès** : L'installateur configure automatiquement un *ServiceAccount* dédié, doté d'un *ClusterRole* aux permissions strictement limitées, limitant ainsi le rayon d'explosion en cas de compromission d'un service.
- **Intégrité du stockage** : L'automatisation prend en charge la gestion des droits sur les volumes persistants (ex: `chown -R 1000:1000` via *initContainer*), éliminant les erreurs d'accès disque courantes dans les déploiements K8s.

3. Architecture Backend et Sécurité Applicative

3.1 Un socle technique asynchrone (FastAPI & Pydantic)

Le cœur de K-Guard repose sur **FastAPI (v0.109.2)**, un choix motivé par sa gestion native des opérations asynchrones et sa haute performance en environnement conteneurisé. Pour garantir une intégrité totale des échanges, nous avons systématisé l'utilisation des modèles **Pydantic** (*ScanRequest*, *PatchImagePayload*) pour l'ensemble des requêtes entrantes. Cette validation stricte des schémas constitue notre première ligne de défense contre les injections et les erreurs de données malveillantes.

3.2 Resilience SRE : Gestion de la Charge et Timeouts

Le traitement des vulnérabilités (notamment les scans Trivy, absents dans cette nouvelle version mais qui a motivé la configuration initiale de K-Guard et dont le base de donnée sert aujourd'hui à la nouvelle stack Falco/ELK) est une opération extrêmement gourmande en ressources CPU et I/O. Pour éviter les instabilités critiques telles que les erreurs **502 Bad Gateway** ou **504 Gateway Timeout** au niveau de l'Ingress Nginx, l'architecture a été conçue pour dissocier les traitements :

- **Background Tasks** : Les tâches lourdes sont systématiquement déléguées aux *Background Tasks* de FastAPI, permettant une réponse immédiate de l'API à l'utilisateur tout en garantissant l'exécution complète du scan en arrière-plan.
- **Persistance Intelligente** : Les résultats sont persistés dans une base **SQLite** montée sur un volume K8s dédié, garantissant la persistance des données d'audit même en cas de redémarrage des pods.

3.3 Sécurité des Accès (Authentication & RBAC)

La sécurité des accès est traitée avec le même niveau d'exigence que l'infrastructure réseau :

- **Flux OAuth2 & JWT** : L'authentification repose sur une implémentation sécurisée du flux OAuth2. L'émission des JSON Web Tokens (JWT) via `python-jose` utilise l'algorithme **HS256**, avec une durée d'expiration restreinte à 600 minutes pour limiter la fenêtre d'exposition en cas de compromission d'un jeton.
- **Principe du Moindre Privilège** : K-Guard applique strictement le principe du moindre privilège. Chaque instance tourne en tant qu'utilisateur non-root (`runAsUser : 1000`), renforçant l'isolation du processus par rapport à l'hôte.

3.4 Sécurité de la "Supply Chain" logicielle

Le backend joue un rôle pivot dans la surveillance de la *Supply Chain*. Il ne se contente pas de scanner, il intervient :

- **Validation des Images** : Chaque image déployée est analysée pour détecter les vulnérabilités avant toute mise en production.
- **Smart Patching** : En cas de détection de faille, l'API utilise l'API Kubernetes via *Strategic Merge Patch* pour injecter la version corrigée directement dans le conteneur cible, garantissant une remédiation rapide sans perturber le déploiement.

4. Module "K-Guard Network Sentinel"

4.1 Gouvernance et Micro-segmentation "Zero Trust"

Le module *Network Sentinel* marque un tournant décisif dans l'architecture de K-Guard. Il transforme le cluster en un environnement "Zero Trust" où la confiance ne peut être présumée. Par défaut, le module applique une politique **Deny-All** sur les namespaces protégés, bloquant l'intégralité du trafic Ingress et Egress. Ce périmètre sécurisé impose une définition explicite des flux autorisés, réduisant ainsi drastiquement la surface d'attaque en cas de mouvement latéral au sein du cluster.

4.2 Hardening Dynamique et Flux Chirurgicaux

Contrairement aux politiques réseau statiques qui deviennent rapidement des goulets

d'étranglement, *Network Sentinel* propose un filtrage intelligent et adaptatif :

- **Adaptation aux flux externes** : Le module restreint l'Ingress Nginx pour n'autoriser exclusivement que les plages d'IP officielles de Cloudflare. Ces plages sont requêtées et mises à jour dynamiquement via l'API, garantissant une protection pérenne contre le contournement des proxies.
- **Segmentation des flux applicatifs** : L'approche permet d'ouvrir des communications de manière chirurgicale. Un exemple concret est l'autorisation exclusive du port TCP 3310 du pod ClamAV, garantissant que seul le frontend du blog puisse interroger le service d'analyse antivirus, isolant totalement le reste du cluster de cette interaction.

4.3 Pilotage et Observabilité Réseau

Sentinel ne se contente pas d'exécuter des règles, il offre une visibilité totale sur l'état de santé du réseau via le dashboard [SentinelView](#) :

- **Score de Sécurité** : Un algorithme calcule un score de 0 à 100 % basé sur le ratio entre les pods durcis (*hardened*) et les pods vulnérables, offrant une métrique de pilotage immédiate pour le responsable sécurité.
- **Topologie Interactive** : Le module génère une représentation visuelle dynamique des flux (edges) entre les pods. Cette vue permet d'identifier instantanément les connexions non autorisées ou les anomalies de communication au sein du maillage Kubernetes.
- **Actions Opérationnelles** : L'interface permet aux administrateurs de déployer ou révoquer les politiques de durcissement en un clic, offrant une flexibilité totale lors des phases de maintenance ou de déploiement d'urgence.

4.4 Automatisation via Ansible

La gouvernance réseau est orchestrée par un moteur Ansible, assurant que chaque règle de filtrage est appliquée uniformément sur l'ensemble du cluster. Cette approche **Policy-as-Code** permet de versionner les politiques réseau dans le dépôt Git du projet, garantissant une traçabilité totale des changements et une reproductibilité parfaite de l'état de sécurité du cluster.

5. Observabilité et Maintenance (SRE)

5.1 Pilotage Opérationnel par le Dashboard "Settings"

L'interface de gestion **Settings** ne constitue pas une simple page de configuration, mais un véritable centre de contrôle de l'infrastructure. Elle centralise les métriques critiques et les commandes de maintenance, permettant à l'administrateur système de maintenir la stabilité du cluster sans recours systématique à la ligne de commande.

5.2 Monitoring SRE : Prévention et Disponibilité

Pour garantir la stabilité du cluster et éviter les arrêts impromptus (OOM Kills), K-Guard intègre une pile d'observabilité mature :

- **Métriques Temps Réel** : Utilisation de **NodeExporter** pour le monitoring de l'état de santé des nœuds hôtes et de **cAdvisor** pour la collecte granulaire des métriques de conteneurs (consommation RAM/CPU).
- **Capacity Planning** : Ces données permettent d'ajuster les **Limits** et **Requests** en temps réel, prévenant ainsi la saturation mémoire qui reste une cause majeure d'instabilité dans les environnements Kubernetes légers.

5.3 Maintenance Active : Gestion du Cycle de Vie des Données

Un défi majeur de l'utilisation de scanners de vulnérabilités est la gestion de leur cache, qui peut rapidement saturer les volumes persistants (PVC). K-Guard implémente :

- **Purge Intelligente** : Une fonctionnalité de purge dédiée aux caches **Trivy** permet de libérer l'espace disque de manière contrôlée, garantissant que le système reste performant sans intervention manuelle lourde sur les volumes.
- **Persistance optimisée** : Cette maintenance complète l'usage du volume **trivy-cache-pvc** (5Gi), assurant un équilibre entre la vitesse des scans (db locale) et la préservation des ressources disque.

5.4 Alerting SecOps : Intégration Cisco Webex

L'observabilité est inutile sans une boucle de rétroaction rapide. K-Guard transforme les événements de sécurité en actions concrètes via son module `CiscoWebexNotifier` :

- **Notifications Temps Réel** : Dès que le moteur Trivy détecte des vulnérabilités de criticité **CRITICAL** ou **HIGH**, le système déclenche automatiquement une alerte structurée vers une Room SOC dédiée via l'API Webex (<https://webexapis.com/v1/messages>)(<https://webexapis.com/v1/messages>)).
- **Configuration Dynamique** : Les paramètres de connexion (Token, Room ID) sont modifiables à la volée depuis le dashboard `Settings` et persistés dans la base SQLite, offrant une agilité maximale aux équipes de sécurité lors d'une phase de réponse à incident (IR).

6. Conclusion et Posture Professionnelle

6.1 Synthèse de l'Architecture

Le projet K-Guard dépasse le stade du simple outil de sécurité pour devenir une plateforme d'ingénierie cohérente et industrialisée. En articulant des briques technologiques modernes (Go, FastAPI, Vue.js, Ansible) autour d'une approche **DevSecOps**, nous avons démontré la faisabilité d'une défense active et automatisée au sein de clusters Kubernetes légers. De l'installation idempotente gérée par binaire Go à la remédiation en temps réel des vulnérabilités, chaque composant a été conçu pour maximiser la résilience opérationnelle tout en minimisant la surface d'exposition.

6.2 Une réponse aux enjeux du marché

K-Guard apporte des solutions concrètes aux problématiques actuelles des entreprises :

- **Réduction du MTTR (Mean Time To Remediation)** : Grâce à l'automatisation des patches et à l'intégration Cisco Webex, les équipes SOC disposent d'un temps de réaction optimisé.
- **Maîtrise du "Day 2 Operations"** : L'intégration des outils de monitoring (cAdvisor, NodeExporter) et de gestion de cache (Trivy) prouve une capacité à gérer la stabilité du

cycle de vie des applications sur le long terme.

- **Sécurité Zero Trust** : Le module *Network Sentinel* valide une compréhension approfondie des enjeux de micro-segmentation dynamique dans des environnements conteneurisés.

6.3 Posture d'Ingénieur et Perspectives

En tant qu'Apprenti Administrateur Systèmes & Réseaux avec une spécialisation DevSecOps, mon ambition au travers de K-Guard est double :

1. **Démontrer une rigueur technique opérationnelle** : La maîtrise de concepts avancés (Strategic Merge Patch, gestion des *AbortController* pour la gestion mémoire, validation de schémas Pydantic) atteste d'une capacité à concevoir des systèmes prêts pour des environnements critiques.
2. **Affirmer une culture d'ingénierie continue** : Mon parcours, orienté vers l'obtention de la certification Cisco DevNet, se reflète dans l'automatisation de bout en bout de l'infrastructure et la promotion d'une culture où la sécurité est intégrée dès le code.

K-Guard n'est pas seulement une application ; c'est la preuve de ma capacité à concevoir, sécuriser et maintenir des infrastructures complexes. Ce projet constitue le socle de mes futures missions en entreprise, où j'aurai à cœur de déployer cette vision de la sécurité active pour garantir la continuité et la résilience des services numériques.